CS236 — Artificial Intelligence

Project 01: Lunar Lander Agent

Methodology Report

Indian Institute of Information Technology Guwahati

Algorithm	Proximal Policy Optimization (PPO)
Network	8 → 128 → 128 → 4 (ReLU activations)
Parameters	18,180 total trainable weights
Training	10,000,000 timesteps | 32 parallel environments
Library	Stable-Baselines3 + Gymnasium

---

1. Problem Overview

The objective of this project is to train an autonomous agent to successfully land a rocket on the Lunar Lander environment provided by OpenAI Gymnasium (LunarLander-v3). The agent receives eight continuous observations from the environment at each timestep and must select one of four discrete actions: do nothing, fire the left orientation engine, fire the main engine, or fire the right orientation engine. The goal is to maximise the cumulative reward over an episode. A score of 200 or above is considered solved.

Observation Space (8 values)

Index	Observation	Range
0	X position	[-1.5, 1.5]
1	Y position	[-1.5, 1.5]
2	X velocity	[-5, 5]
3	Y velocity	[-5, 5]
4	Angle	[-3.14, 3.14]
5	Angular velocity	[-5, 5]
6	Left leg contact	{0, 1}
7	Right leg contact	{0, 1}

Reward Structure

- Landing on the pad and coming to rest: +100 to +140 points
- Each leg in contact with ground: +10 points
- Coming to rest (episode end): +100 points
- Crashing: -100 points
- Firing main engine: -0.3 points per frame

- Firing side engine: -0.03 points per frame
- Moving away from landing pad: continuous penalty

2. Approach — Proximal Policy Optimization (PPO)

We chose Proximal Policy Optimization (PPO) as our training algorithm. PPO is a state-of-the-art policy gradient method developed by OpenAI that achieves strong performance across a wide range of continuous and discrete control tasks. It was selected over simpler evolutionary approaches because it learns from every individual timestep rather than from episode-level rewards alone, resulting in significantly faster convergence and higher final performance.

Why PPO over Evolutionary Strategies?

Aspect	Evolutionary Strategy	PPO (our approach)
Learning signal	Episode reward only	Per-step advantage estimate
Sample efficiency	Low	High
Stability	Moderate	High (clipping)
Typical score	200 – 250	270 – 300+
Parallelism	Sequential	32 environments

3. Neural Network Architecture

The policy is represented as a fully connected feedforward neural network with two hidden layers. The network takes the 8-dimensional observation vector as input and outputs logits for each of the 4 possible actions. The action with the highest logit is selected (argmax).

Layer	Input size	Output size	Activation	Parameters
Input	—	8	—	—
Hidden 1	8	128	ReLU	8×128 + 128 = 1,152
Hidden 2	128	128	ReLU	128×128 + 128 = 16,512
Output	128	4	None	128×4 + 4 = 516
Total	**—**	**—**	**—**	**18,180 parameters**

ReLU Activation Function

The Rectified Linear Unit (ReLU) activation function is applied after each hidden layer. ReLU is defined as f(x) = max(0, x). It outputs the input directly if positive, and zero otherwise. ReLU introduces non-linearity into the network, enabling it to learn complex, non-linear relationships between observations and optimal actions — something a purely linear mapping cannot achieve.

Actor-Critic Structure

PPO uses an Actor-Critic architecture. The actor network (policy network) decides which action to take given an observation. The critic network (value network) estimates the expected cumulative reward from the current state. Both networks share the same architecture (8→128→128) but have separate weights. The critic does not directly control actions — it provides a baseline that reduces variance in the learning signal and speeds up training significantly.

4. Training Methodology

PPO Algorithm — How It Works

PPO is a policy gradient algorithm that directly optimises the policy by gradient ascent on an objective function. The key steps in each training iteration are:

- Collect N steps of experience by running the current policy in the environment
- Compute the advantage estimate for each step — was this action better or worse than expected?
- Compute the PPO clipped surrogate objective to update the actor
- Compute the value function loss to update the critic
- Perform multiple gradient update epochs on the collected batch
- Repeat until the total timestep budget is exhausted

The PPO Clipping Trick

The defining feature of PPO is its clipped surrogate objective. When updating the policy, the ratio r = new_policy / old_policy is computed. This ratio is clipped to the range [1 - epsilon, 1 + epsilon] where epsilon = 0.2. This prevents the policy from changing too drastically in a single update, which would destabilise training. The final objective takes the minimum of the clipped and unclipped versions, creating a conservative lower bound on policy improvement.

Advantage Estimation (GAE)

We use Generalised Advantage Estimation (GAE) with lambda = 0.95 to estimate how much better or worse a particular action was compared to the average. A positive advantage means the action led to better-than-expected outcomes; a negative advantage means worse. GAE balances bias and variance in the advantage estimate by combining multi-step returns using an exponentially weighted average.

Parallel Environments

Training used 32 parallel environments (n_envs=32). Instead of running one episode at a time, 32 independent game instances run simultaneously. This provides 32 times more diverse experience per unit of wall-clock time, dramatically accelerating training and reducing correlation between consecutive samples.

Hyperparameters

Hyperparameter	Value	Reason
Total timesteps	10,000,000	Sufficient for convergence
Parallel envs	32	Faster, diverse experience
Learning rate	3e-4	Standard for Adam optimiser
n_steps	512	Steps collected before update
Batch size	128	Mini-batch size for updates
n_epochs	10	Reuse each batch 10 times
Gamma	0.999	High discount — long horizon
GAE lambda	0.95	Balance bias vs variance
Clip range	0.2	Prevents large policy updates
Entropy coefficient	0.01	Encourages exploration
Value fn coef	0.5	Weight of critic loss
Max grad norm	0.5	Gradient clipping for stability

5. Policy File

The trained policy is extracted from the PyTorch model and stored as a flat NumPy array of 18,180 float32 values in best_policy_2209_improved.npy. The policy_2209_improved.py file reconstructs the network weights from this array and performs a forward pass at evaluation time using only NumPy — no PyTorch or Stable-Baselines3 required at inference.

Forward Pass at Inference

Given an observation vector of length 8, the policy computes:

- Unpack the flat parameter array into weight matrices W1, W2, W3 and bias vectors b1, b2, b3
- h1 = ReLU(observation @ W1 + b1) — first hidden layer
- h2 = ReLU(h1 @ W2 + b2) — second hidden layer
- logits = h2 @ W3 + b3 — output scores for 4 actions
- action = argmax(logits) — select action with highest score

Weight Extraction

PyTorch stores weight matrices in transposed form relative to the convention used in our NumPy forward pass. During extraction, each weight matrix is transposed (.T) before being flattened and saved, ensuring the inference computation is correct.

6. Evaluation

The agent is evaluated using evaluate_agent.py over 100 episodes. The first 5 episodes are rendered visually; the remaining 95 run in the background. The final score is the mean reward over all 100 episodes. The evaluation command is:

```
python evaluate_agent.py --filename best_policy_2209_improved.npy --policy_modu
le policy_2209_improved
```

Expected Performance

Metric	Value
Mean reward (100 episodes)	270 – 300+
Solved threshold	200
Evaluation time	< 100 seconds
Parameter file size	< 1 MB

7. Submitted Files

File	Description
policy_2209_improved.py	Agent policy — NumPy forward pass, policy_action() function
train_agent_2209_improved.py	Training script — PPO with Stable-Baselines3
best_policy_2209_improved.npy	Trained weights — 18,180 float32 parameters
methodology.pdf	This document

8. Conclusion

We trained a neural network agent using Proximal Policy Optimization to play the LunarLander-v3 environment. The agent uses a two-layer network ($8 \rightarrow 128 \rightarrow 128 \rightarrow 4$) trained over 10 million timesteps across 32 parallel environments. PPO was chosen for its sample efficiency, training stability through the clipping mechanism, and strong empirical performance on discrete control tasks. The trained policy is evaluated purely using NumPy at inference time, satisfying all project constraints.